DSCI-D 532: Applied Database Technologies

Instructor: Dr. Scrivner

Wyatt Keller

July 31, 2026

QuickCast: Final Project Report

Application Repository

The QuickCast repository, which includes all source code, documentation, and a README, is publicly available at: github.com/thatguy-jpeg/QuickCast

Project Overview

QuickCast is a business intelligence tool that gives users quick visual overviews of their sales data. It generates charts across four dimensions, being employees, locations, products, and customers, and includes a “forecast” feature that graphs order history for a specific product across every date it appears in the database. QuickCast is built to be simple, with almost every interaction happening through buttons or counters, with manual typing required only to enter a product name for the forecast feature or a username for account creation.

The backend pairs a SQLite database with Flask, which pairs nicely with the rest of the Python stack. SQLite's lightweight and file-based design suited the free-tier hosting used for this project. Each button in the UI maps to a SQL query with different conditions, passed as arguments to the backend. This logic runs throughout the query-handling scripts, and together the buttons support all CRUD operations. The frontend is hosted on GitHub Pages and calls a backend hosted on PythonAnywhere, which is a free pairing, but sadly does not support heavy workloads. QuickCast is only intended for demo and portfolio purposes and it does not have the capacity or security needed for handling real user data.

Changes from the Original Proposal

- The forecast feature does not project future quantities as originally proposed, instead it visualizes historical sales over time.
- The password field went unused and the application does not implement any user authentication.
- Several fields created in the database schema went unused by the final queries.
- The backend is hosted on PythonAnywhere instead of an Ngrok tunnel to a local laptop, as originally planned.

Final Application

Deployed app (GitHub Pages): QuickCast — Retail Sales Terminal

The sections below walk through QuickCast's core features, which are account creation, the query board with its four dimensions and controls, query results, forecasting, record editing, and account deletion.

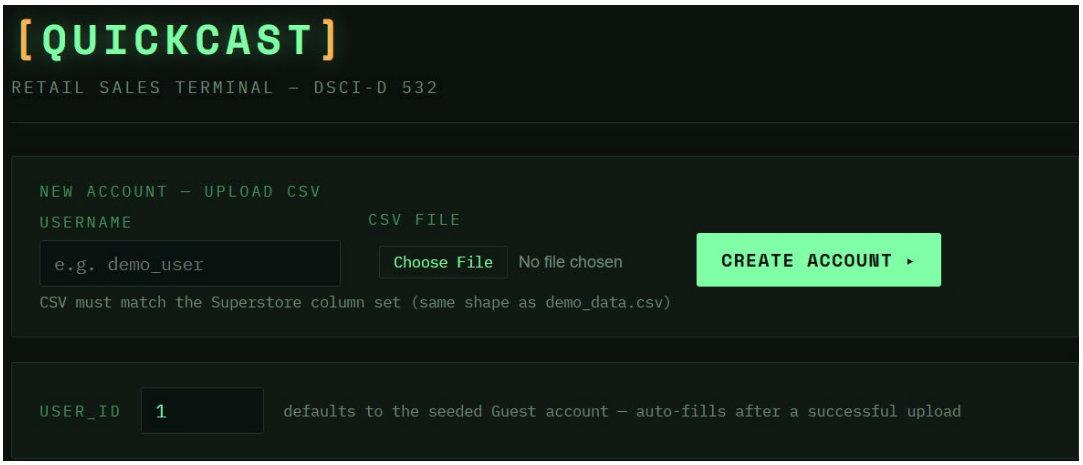

Figure 1. Account creation: enter a username, choose a CSV file, and create an account to receive a new user_id. The field defaults to the seeded Guest account.

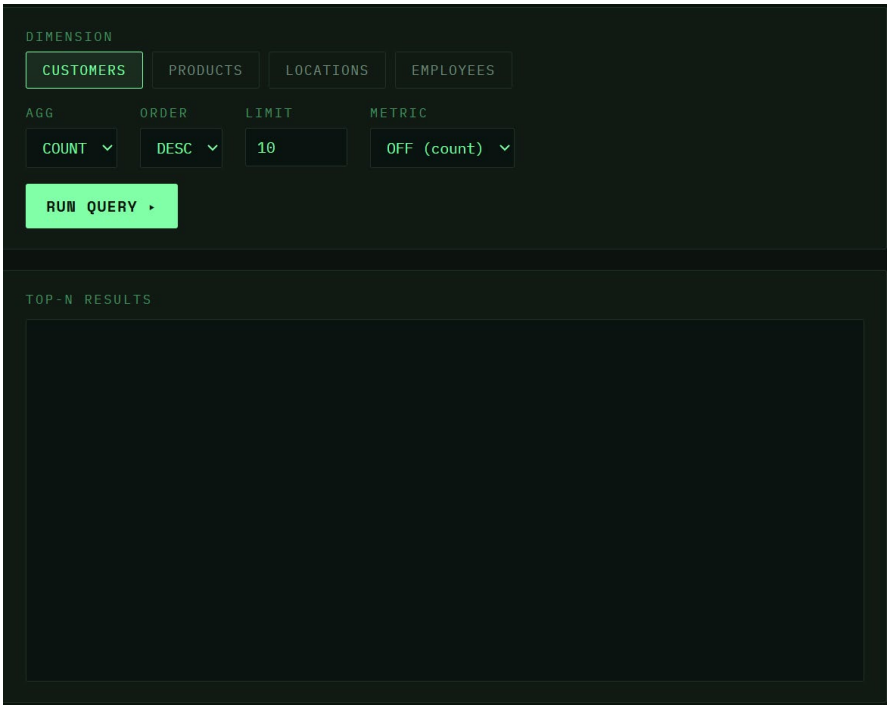

Figure 2. The query board, shown with the Customers dimension selected. Controls include AGG (aggregation), ORDER, LIMIT (result count), and METRIC (count, sales, or profit); results render below in the Top-N Results dashboard.

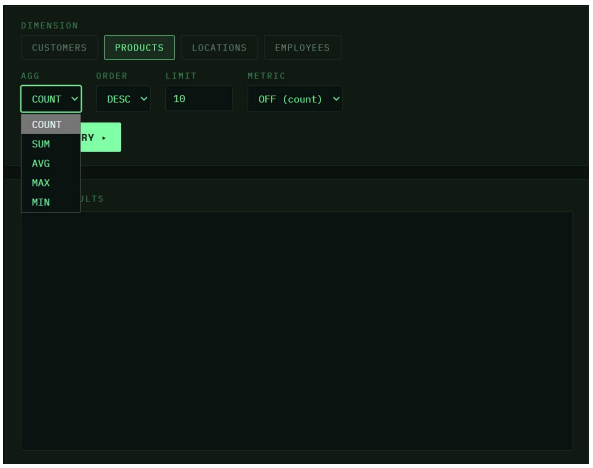

Figure 3. Products dimension, with the AGG dropdown open (COUNT, SUM, AVG, MAX, MIN).

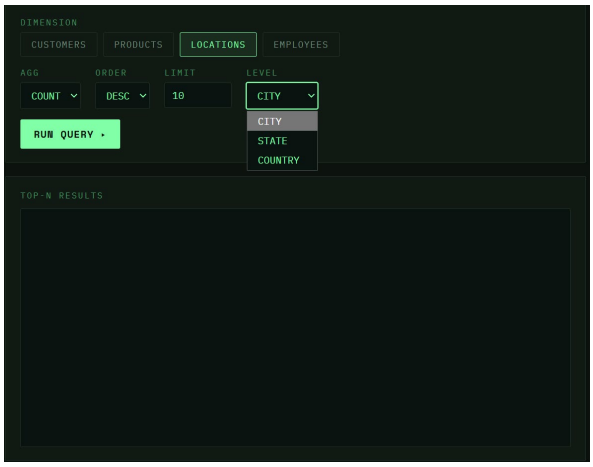

Figure 4. Locations dimension. Here LEVEL replaces METRIC, letting results roll up by city, state, or country.

Additional controls not pictured separately: the Employees dimension uses a METRIC dropdown (off/count, sales, profit); LIMIT is a counter for how many results to return; and ORDER toggles results between descending and ascending.

Figure 5. A successful query for the top 20 products sold, rendered as a bar chart from "Xerox 1881" (highest) down to a Wilson Jones binder (20th).

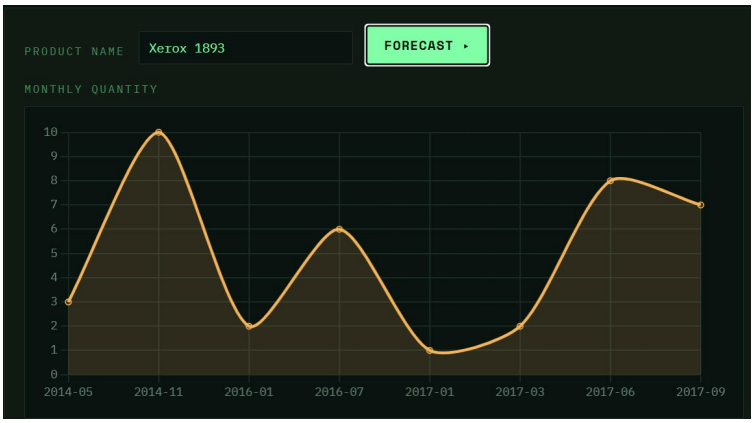

Figure 6. The forecast dashboard after entering "Xerox 1893", showing quantity sold over time for that product.

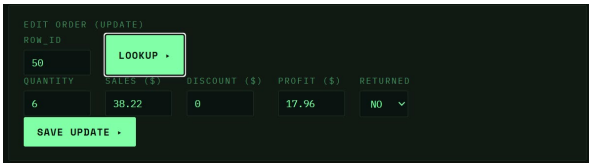

Figure 7. Record editing: LOOKUP prefills a row's values by row_id; fields can then be edited and saved with SAVE UPDATE.

Figure 8. Account deletion: entering a user_id (other than the Guest account) and confirming removes all associated records.

Technical Summary

Technologies Used

- Python — Flask, Flask-CORS, SQLite3, Pandas, Pathlib
- JavaScript — Chart.js (chart rendering), HTML, CSS
- Hosting — PythonAnywhere (backend/API), GitHub Pages (frontend)

app.py runs the backend on PythonAnywhere using Flask and Flask-CORS. It calls into supporting scripts that use SQLite, Pandas, and Pathlib to query the QuickCast database. The GitHub Pages frontend calls that API and renders the returned data as charts using Chart.js.

Team Member Contributions

This was a one-person team. Wyatt Keller produced all code, ideas, and database normalization and design for the project.

AI Assistance Statement

Claude was used to assist development, including code generation, debugging, explaining errors, advising on best practices, improving code quality, and formatting documentation. Furthermore, Claude was used in helping format and fixing prose of this report. Specific files and the nature of assistance:

- **app.js, config.js, index.html, style.css, app.py —** heavily AI-assisted. Claude helped implement the frontend, the Flask backend, and the API calls between the two servers.
- **insertion.py —** Claude assisted with the Pandas merge logic used to map IDs to the correct records. The insertion SQL queries themselves were not AI-generated.
- **query_functions.py —** Claude assisted with debugging query parameterization, as in passing arguments into functions and routing them to the correct query without crashes.

All database normalization, database design, schema creation, and base SQL queries were created without AI assistance. Full AI conversation logs are available upon request.

Reflection

What Worked Well

The main thing that worked well for me was the original idea I had actually worked deployment wise. I went in wanting to make it free, and I was able to make it work out! Other thing that worked out was that SQLite ended up being the perfect database to use for me. Out of all the languages we used for this course, I was most comfortable with SQL, but also, I do not have too much experience with linking services. Due to this mix, I wanted to use Python for all that I could, and SQLite being an option available was awesome. I did look into SQLAlchemy with Flask and whatnot, but I thought it would be overkill/unnecessary for the application.

Biggest Challenge, Most Interesting/Rewarding Experience

The biggest challenge would most definitely be making an entire application in itself. I have made backend scripts and functions to work with databases, like for a web scraper or two I made. I've also made a website before, like with my portfolio website. Though, I have not done both of these combined. It was very complicated having to think of how the scripts would be used on the front end while making the backend, because if I didn't, I'd have to redesign everything to work out.

I also do not have experience in working with API's, network addresses, and how to handle the different data types that different applications would send over. For example, the JavaScript front end requires a completely different data type than what the SQLite would pass its way, and so having to convert brought me having to deal with new problems I was not expecting.

What I Would Improve

The main thing I would improve is to ensure the outlook I had at the beginning was properly scoped. I thought I was being conservative with the promises I was making, but even then, I wasn't. I promised some features that just didn't go into production. For example, there were several features like an actual forecasting button, or using discounts, or even heavily complex queries that I was suggesting could be made. While they likely could have been done, I did not have the time or ability to make them.

References

- Retail Supply Chain Sales Dataset (Kaggle), the source data used to seed QuickCast
- PythonAnywhere — Deploying an Existing Project (help docs, used for backend deployment)
- Flask Documentation
- SQLite Documentation
- Chart.js Documentation
- Pandas Documentation
- GitHub Pages Documentation

Personal Note:

Hi Dr. Scrivner and Mr. Jain,

I've really enjoyed this course and everything I've learned through it. It's definitely one of the harder ones that I've taken, but the knowledge was invaluable. I remember starting the course and seeing the SQL lessons and I immediately enjoyed it. I was wanting to learn SQL and so I was overjoyed that this class went in-depth about it.

This project also really helped me develop myself. It took me out of my comfort zone and really improved my skills in a variety of ways. I have a personal interest in the supply chain and so being able to operate a project to specifically work towards that was fulfilling.

Lastly, thank you so much Dr. Scrivner for allowing me to take this course. I'm not sure if you remembered by email back then, but it was me requesting to take this course as an undergraduate. This course brought to me many essential skills that would normally be a blind spot for my undergraduate education. So, this message is for me to express my gratitude.

I hope you all enjoy the rest of your summers and next year! I look forward to possibly having another class with you in the future.

Best regards,
Wyatt Keller